

AML Lab 4

ML Experiments

Simón Ramírez Amaya ¹

¹School of Information, UC Berkeley

February 18 of 2026



ML experiments

Key concepts:

- Parameters (e.g. β in lasso)
- Hyperparameters (e.g. λ in lasso)
- Out-of-sample performance
- Train/test splits
- K-fold cross-validation
- Nested cross-validation

The End Goals:

1. **Given some data... how to choose a model? (deployment task)**
2. **Given some data... how to estimate performance? (evaluation task)**

Today:

- Revisit these concepts/questions in more detail + pseudocode.



Key takeaways

Key takeaways:

1. Evaluation and deployment are different tasks, with different solutions (although they are related).
2. Cross-validation plays a role in both, but for very different reasons.
3. Discipline is paramount, but there is room for creativity and intuition (e.g. hyperparameter space).

Further reading:

- Cawley and Talbot (2010), Wainer and Cawley (2021)
- Patterns, Predictions and Actions - Chapter 6 (Recht and Hardt 2022)
- Insightful Q&A in Cross Validated (Stack Exchange)



Notation

→ Our building blocks:

- $X^{[c \times 1]}$ = "vector of features, length c "
- $y^{[1 \times 1]}$ = "target variable, length 1 "
- $\beta^{[p \times 1]}$ = "vector of parameters, length p "

→ What is a model?

- $m(X, \beta)^{[1 \times 1]}$ = "function that takes features and parameters and returns a real-valued prediction"

→ What info do we have?

- We observe $\{(X_i^{[c \times 1]}, y_i^{[1 \times 1]})\}_{i=1}^N$.
- $X^{[N \times c]}$ = "matrix of stacked features, dimension N by c "
- $Y^{[N \times 1]}$ = "matrix of stacked outcomes, dimension N by 1 "

→ How good is a prediction?

- $loss(\beta, X, Y)^{[1 \times 1]}$ = "function that compares predictions to target and returns a real number representing loss"



Notation

→ Our building blocks:

- $X^{[c \times 1]}$ = "vector of features, length c "
- $y^{[1 \times 1]}$ = "target variable, length 1 "
- $\beta^{[p \times 1]}$ = "vector of parameters, length p "

→ What is a model?

- $m(X, \beta)^{[1 \times 1]}$ = "function that takes features and parameters and returns a real-valued prediction"

→ What info do we have?

- We observe $\{(X_i^{[c \times 1]}, y_i^{[1 \times 1]})\}_{i=1}^N$.
- $X^{[N \times c]}$ = "matrix of stacked features, dimension N by c "
- $Y^{[N \times 1]}$ = "matrix of stacked outcomes, dimension N by 1 "

→ How good is a prediction?

- $loss(\beta, X, Y)^{[1 \times 1]}$ = "function that compares predictions to target and returns a real number representing loss"



Notation

→ Our building blocks:

- $X^{[c \times 1]}$ = "vector of features, length c "
- $y^{[1 \times 1]}$ = "target variable, length 1 "
- $\beta^{[p \times 1]}$ = "vector of parameters, length p "

→ What is a model?

- $m(X, \beta)^{[1 \times 1]}$ = "function that takes features and parameters and returns a real-valued prediction"

→ What info do we have?

- We observe $\{(X_i^{[c \times 1]}, y_i^{[1 \times 1]})\}_{i=1}^N$.
- $\mathbb{X}^{[N \times c]}$ = "matrix of stacked features, dimension N by c "
- $\mathbb{Y}^{[N \times 1]}$ = "matrix of stacked outcomes, dimension N by 1 "

→ How good is a prediction?

- $loss(\beta, X, Y)^{[1 \times 1]}$ = "function that compares predictions to target and returns a real number representing loss"



Notation

→ Our building blocks:

- $X^{[c \times 1]}$ = "vector of features, length c "
- $y^{[1 \times 1]}$ = "target variable, length 1 "
- $\beta^{[p \times 1]}$ = "vector of parameters, length p "

→ What is a model?

- $m(X, \beta)^{[1 \times 1]}$ = "function that takes features and parameters and returns a real-valued prediction"

→ What info do we have?

- We observe $\{(X_i^{[c \times 1]}, y_i^{[1 \times 1]})\}_{i=1}^N$.
- $\mathbb{X}^{[N \times c]}$ = "matrix of stacked features, dimension N by c "
- $Y^{[N \times 1]}$ = "matrix of stacked outcomes, dimension N by 1 "

→ How good is a prediction?

- $loss(\beta, X, Y)^{[1 \times 1]}$ = "function that compares predictions to target and returns a real number representing loss"



Our problem

- We'll find algorithmic solutions that use the data available $(\mathbb{X}, Y)$ for each of the following cases:

	Deployment task	Evaluation task
No hyperparams	$\beta^* = ?$	$I^* = ?$
Yes hyperparams	$\beta^* = ?$	$I^* = ?$



Case I: No hyperparams

→ Highly idealized setting, but useful to build intuition. There exists only one learning procedure:

- $\mathcal{L}(\mathbb{X}_{learn}, Y_{learn})^{[p \times 1]}$ = procedure that takes some learning features and outcomes and returns a parameter value β^{learn} .



Model finding procedure

Algorithm 1 Model finding procedure

Require: $\mathbb{X}_{learn}, Y_{learn}$
 $\beta^{learn} \leftarrow \mathcal{L}(\mathbb{X}_{learn}, Y_{learn})$
return β^{learn}



Our problem

	Deployment task	Evaluation task
No hyperparams	$\beta^* = \text{algo1}(\mathbb{X}, Y)$	?
Yes hyperparams	?	?

→ The final model is estimated using all the data available.



Estimating loss with test/train split

Algorithm 2 Estimating loss with train/test split

Require: $\mathbb{X}_{est}, Y_{est}$

$\mathbb{X}_{train}, Y_{train}, \mathbb{X}_{test}, Y_{test} = split(\mathbb{X}_{est}, Y_{est})$

$\beta^{train} \leftarrow algo1(\mathbb{X}_{train}, Y_{train})$

$J^{test} \leftarrow loss(\beta^{train}, \mathbb{X}_{test}, Y_{test})$

return J^{test}



Our problem

	Deployment task	Evaluation task
No hyperparams	$\beta^* = \text{algo1}(\mathbb{X}, Y)$	$l^* = \text{algo2}(\mathbb{X}, Y)$
Yes hyperparams	?	?

→ In general, $\beta^* \neq \beta^{train}$ computed in Algorithm 2!



Estimating loss with cross-validation

Algorithm 3 Estimating loss with CV

Require: $\mathbb{X}_{est}, Y_{est}, k_{outer}$

$\mathbb{X}_{est,(\cdot)}, Y_{est,(\cdot)} \leftarrow split(\mathbb{X}_{est}, Y_{est}, k_{outer})$

for $k \in 1, \dots, k_{outer}$ **do**

$l_k^* \leftarrow loss(algo1(\mathbb{X}_{est,(-k)}, Y_{est,(-k)}), \mathbb{X}_{est,(k)}, Y_{est,(k)})$

end for

$L^* = [l_1^*, \dots, l_{k_{outer}}^*]$

return L^*



Our problem

	Deployment task	Evaluation task
No hyperp	$\beta^* = \text{algo1}(\mathbb{X}, Y)$	$I^* = \text{algo2}(\mathbb{X}, Y)$ or $L^* = \text{algo3}(\mathbb{X}, Y)$
Yes hyperp	?	?



Case II: Hyperparams

→ Now, let's think of a setting where there is not a single way of learning, but many.

- $H^{[h \times 1]}$ = "vector of hyperparameters, length h "
- $\mathcal{H}$ = "set of all possible hyperparameters, finite"
- $\mathcal{L}(\mathbb{X}_{learn}, Y_{learn}, H)^{[p \times 1]}$ = procedure that takes learning features, outcomes and a hyperparameter and returns a parameter value β^*



Model finding procedure with hyperparams

Algorithm 4 Model finding procedure with hyperparams

Require: $\mathbb{X}_{learn}, Y_{learn}, k_{inner}$
 $\mathbb{X}_{learn,(\cdot)}, Y_{learn,(\cdot)} \leftarrow \text{split}(\mathbb{X}_{learn}, Y_{learn}, k_{inner})$
for $k \in 1, \dots, k_{inner}$ **do**
 for $h \in \mathcal{H}$ **do**
 $\beta_{k,h} \leftarrow \mathcal{L}(\mathbb{X}_{learn,(-k)}, Y_{learn,(-k)}, h)$
 $l_{k,h} \leftarrow \text{loss}(\beta_{k,h}, \mathbb{X}_{learn,(k)}, Y_{learn,(k)})$
 end for
end for
 $h^* \leftarrow \text{argmin}_{\mathcal{H}} [\frac{1}{k_{inner}} \sum_{k=1}^{k_{inner}} l_{k,h}]$
 $\beta^* \leftarrow \mathcal{L}(\mathbb{X}_{learn}, Y_{learn}, h^*)$
return β^*

→ Bit more complicated than Algorithm 1, since we use cross-validation to learn the optimal hyperparams.



Our problem

	Deployment task	Evaluation task
No hyperp	$\beta^* = \text{algo1}(\mathbb{X}, Y)$	$I^* = \text{algo2}(\mathbb{X}, Y)$ or $L^* = \text{algo3}(\mathbb{X}, Y)$
Yes hyperp	$\beta^* = \text{algo4}(\mathbb{X}, Y)$	?



Estimating loss with train/test split (hyperparams)

Algorithm 5 Estimating loss with train/test split (hyperparams)

Require: $\mathbb{X}_{est}, Y_{est}, k_{inner}$

$\mathbb{X}_{train}, Y_{train}, \mathbb{X}_{test}, Y_{test} = \text{split}(\mathbb{X}_{est}, Y_{est})$

$\beta^* \leftarrow \text{algo4}(\mathbb{X}_{train}, Y_{train}, k_{inner})$

$l^* \leftarrow \text{loss}(\beta^*, \mathbb{X}_{test}, Y_{test})$

return l^*



Estimating loss with CV

Algorithm 6 Estimating out-of-sample loss with CV (hyperparams)

Require: $\mathbb{X}_{est}, Y_{est}, k_{inner}, k_{outer}$

$\mathbb{X}_{est,(\cdot)}, Y_{est,(\cdot)} \leftarrow \text{split}(\mathbb{X}_{est}, Y_{est}, k_{outer})$

for $k \in 1, \dots, k_{outer}$ **do**

$l_k^* \leftarrow \text{loss}(\text{algo4}(\mathbb{X}_{est,(-k)}, Y_{est,(-k)}, k_{inner}), \mathbb{X}_{est,(k)}, Y_{est,(k)})$

end for

$L^* = [l_1^*, \dots, l_{k_{folds_outer}}^*]$

return L^*



Our problem

	Deployment task	Evaluation task
No hyperp	$\beta^* = \text{algo1}(\mathbb{X}, Y)$	$I^* = \text{algo2}(\mathbb{X}, Y)$ or $L^* = \text{algo3}(\mathbb{X}, Y)$
Yes hyperp	$\beta^* = \text{algo4}(\mathbb{X}, Y)$	$I^* = \text{algo5}(\mathbb{X}, Y)$ or $L^* = \text{algo6}(\mathbb{X}, Y)$



What if there are multiple candidate models?

Algorithm 7 Model finding procedure

Require: $\mathbb{X}_{learn}, Y_{learn}, k_{inner}$

$\mathbb{X}_{learn}, (\cdot), Y_{learn}, (\cdot) \leftarrow \text{split}(\mathbb{X}_{learn}, Y_{learn}, k_{inner})$

for $m \in \mathcal{M}$ **do**

for $k \in 1, \dots, k_{inner}$ **do**

for $h \in \mathcal{H}_m$ **do**

$\beta_{m,k,h} \leftarrow \mathcal{L}_m(\mathbb{X}_{learn,(-k)}, Y_{learn,(-k)}, h)$

$l_{m,k,h} \leftarrow \text{loss}(\beta_{m,k,h}, \mathbb{X}_{learn,(k)}, Y_{learn,(k)})$

end for

end for

end for

$m^*, h_m^* \leftarrow \text{argmin}_{\mathcal{M}, \mathcal{H}} [\frac{1}{k_{inner}} \sum_{k=1}^{k_{inner}} l_{m,k,h}]$

$\beta^* \leftarrow \mathcal{L}_{m^*}(\mathbb{X}_{learn}, Y_{learn}, h_m^*)$

return β^*

→ In PS5, you'll implement this algorithm using tools from scikit-learn.

